High-order estimation-based properties and high-order observers for labeled finite-state automata

Computing "know what" in a sequence of labeled finite-state automata

Kuize Zhang[1] Xiaoguang Han[2] Carla Seatzu[3] Alessandro Giua[3]

[1]School of Mathematics and Statistics
Xi'an Jiaotong University, China
kuize.zhang@xjtu.edu.cn

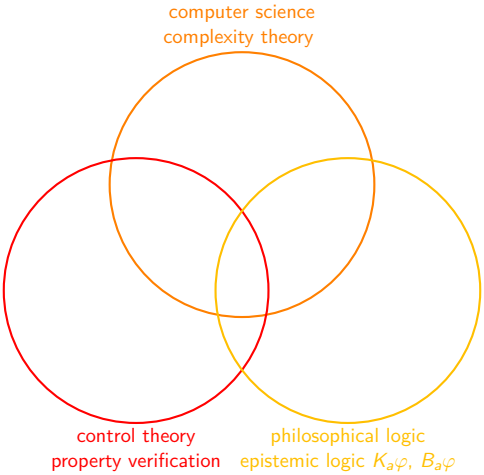
[2]College of Electronic Information and Automation
Tianjin University of Science and Technology, China

[3]Department of Electrical and Electronic Engineering
University of Cagliari, Italy

15 June 2026

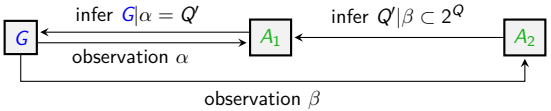
Content

1. Motivation
2. Review of basic tools — concurrent composition and observer
3. A system consisting of a single agent
   - A general framework
   - Case studies
4. A system consisting of two ordered agents
   - A general framework
   - Case studies
5. A system consisting of a finite number of ordered agents
   - A general framework
   - Case studies
6. Conclusion


computer science
complexity theory
control theory
property verification
philosophical logic
epistemic logic $K_a\varphi$, $B_a\varphi$

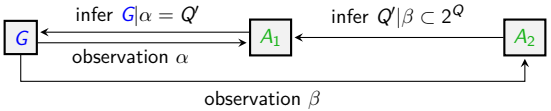
Consider the following scenario consisting of an operating system G and agents A_1 and A_2 who are working on G. Both A_1 and A_2 can observe partial information of G. Assume A_2 knows the part of G that can be observed by A_1.

A_1 wants to estimate the states of G based on its observations to G. A_2 **wants to know whether A_1 can uniquely determine the state of G**, based on A_2's observations to G, by inferring A_1's estimates to the states of G.



Consider the following scenario consisting of an operating system G and agents A_1 and A_2 who are working on G. Both A_1 and A_2 can observe partial information of G. Assume A_2 knows the part of G that can be observed by A_1.

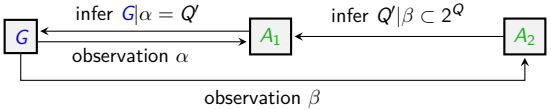
A_1 wants to estimate the states of G based on its observations to G. A_2 **wants to know whether A_1 can uniquely determine the state of G**, based on A_2's observations to G, by inferring A_1's estimates to the states of G.



When G is modeled as a partially-observed finite-state transition system, how to formulate the scenario and how to solve the problem algorithmically?

Consider the following scenario consisting of an operating system G and agents A_1 and A_2 who are working on G. Both A_1 and A_2 can observe partial information of G. Assume A_2 knows the part of G that can be observed by A_1.

A_1 wants to estimate the states of G based on its observations to G. A_2 **wants to know whether A_1 can uniquely determine the state of G**, based on A_2's observations to G, by inferring A_1's estimates to the states of G.



When G is modeled as a partially-observed finite-state transition system, how to formulate the scenario and how to solve the problem algorithmically?

How to formulate and computing "**know what**" in finite transition systems?

Content

1. Motivation
2. Review of basic tools — concurrent composition and observer
3. A system consisting of a single agent
   - A general framework
   - Case studies
4. A system consisting of two ordered agents
   - A general framework
   - Case studies
5. A system consisting of a finite number of ordered agents
   - A general framework
   - Case studies
6. Conclusion

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

Definition 1

A deterministic finite automaton (DFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$,

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

Definition 1

A deterministic finite automaton (DFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$,

- Q is a finite set of states,

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

Definition 1

A deterministic finite automaton (DFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$,

- Q is a finite set of states,
- Σ is an alphabet,

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

Definition 1

A deterministic finite automaton (DFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$,

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

Definition 1

A deterministic finite automaton (DFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$,

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the initial state (aka start state), and

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

Definition 1

A deterministic finite automaton (DFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$,

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the initial state (aka start state), and
- $F \subset Q$ is the set of final states (aka accept states).

- Σ: alphabet, a nonempty and finite set such that every sequence of elements of Σ is a unique sequence of elements of Σ
- e.g., $\{0, 1\}$ and $\{0, 01\}$ are alphabets, but $\{0, 00\}$ isn't, because $000 = 0\ 00 = 00\ 0$
- Σ^*: set of words over Σ, including the empty word ϵ

Definition 1

A deterministic finite automaton (DFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$,

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the initial state (aka start state), and
- $F \subset Q$ is the set of final states (aka accept states).

transition $q \xrightarrow{\sigma} q'$: when the DFA is in state q and reads letter σ, it transitions to state q'

The transition function δ is recursively extended to $Q \times \Sigma^* \to Q$ as usual: for all $q \in Q$, $s \in \Sigma^*$, and $a \in \Sigma$, $\delta(q, sa) = \delta(\delta(q, s), a)$.

The transition function δ is recursively extended to $Q \times \Sigma^* \to Q$ as usual: for all $q \in Q$, $s \in \Sigma^*$, and $a \in \Sigma$, $\delta(q, sa) = \delta(\delta(q, s), a)$.

Definition 2

Let Σ be an alphabet and $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA. A word $w \in \Sigma^*$ is called accepted by DFA A if $\delta(q_0, w) \in F$. The formal language $L(A)$ recognized by DFA A is the set of all words accepted by A, i.e., $\{w \in \Sigma^* | \delta(q_0, w) \in F\}$. A formal language recognized by some DFA is called regular.

The transition function δ is recursively extended to $Q \times \Sigma^* \to Q$ as usual: for all $q \in Q$, $s \in \Sigma^*$, and $a \in \Sigma$, $\delta(q, sa) = \delta(\delta(q, s), a)$.

Definition 2

Let Σ be an alphabet and $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA. A word $w \in \Sigma^*$ is called accepted by DFA A if $\delta(q_0, w) \in F$. The formal language $L(A)$ recognized by DFA A is the set of all words accepted by A, i.e., $\{w \in \Sigma^* | \delta(q_0, w) \in F\}$. A formal language recognized by some DFA is called regular.

Example 3

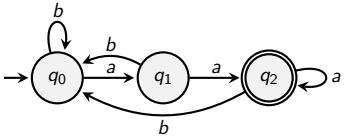


Figure 1: A DFA that accepts exactly all words ending with *aa*.

Definition 4

A nondeterministic finite automaton (NFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where

Definition 4

A nondeterministic finite automaton (NFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states,

Definition 4

A nondeterministic finite automaton (NFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states,
- Σ is an alphabet,

Definition 4

A nondeterministic finite automaton (NFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to 2^Q$ is the transition function,

Definition 4

A nondeterministic finite automaton (NFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to 2^Q$ is the transition function,
- $q_0 \in Q$ is the initial state, and

Definition 4

A nondeterministic finite automaton (NFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to 2^Q$ is the transition function,
- $q_0 \in Q$ is the initial state, and
- $F \subset Q$ is the set of final states.

Definition 4

A nondeterministic finite automaton (NFA) is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \to 2^Q$ is the transition function,
- $q_0 \in Q$ is the initial state, and
- $F \subset Q$ is the set of final states.

Definition 5

Let Σ be an alphabet and $A = (Q, \Sigma, \delta, q_0, F)$ be an NFA. A word $w \in \Sigma^*$ is called accepted by A if $\delta(q_0, w) \cap F \neq \emptyset$. The formal language recognized by A is the set of all words accepted by A, i.e., $\{w \in \Sigma^* | \delta(q_0, w) \cap F \neq \emptyset\}$.

Example 6

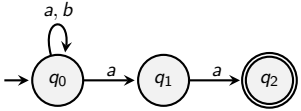


Figure 2: An NFA that accepts exactly all words ending with *aa*.

Example 6

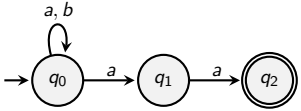


Figure 2: An NFA that accepts exactly all words ending with *aa*.

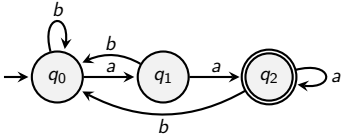


Figure 3: An equivalent DFA (as shown in Figure 1).

- In labeled finite-state automata, concurrent composition (ConCom) provides a unified method to verifying almost all inference-based properties (e.g., detectability, diagnosability, and predictability), and removes two fundamental assumptions of deadlock-freeness and divergence-freeness used for nearly 30 years;

[a] K. Zhang (2023a). "A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems". In: *Annual Reviews in Control* 56, 100902(1–21).

- In labeled finite-state automata, concurrent composition (ConCom) provides a unified method to verifying almost all inference-based properties (e.g., detectability, diagnosability, and predictability), and removes two fundamental assumptions of deadlock-freeness and divergence-freeness used for nearly 30 years;
- ConCom + observer provide the most efficient algorithms for verifying concealment-based properties (e.g., all kinds of state-based opacity)[a].

[a] K. Zhang (2023a). "A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems". In: *Annual Reviews in Control* 56, 100902(1–21).

- In labeled finite-state automata, concurrent composition (ConCom) provides a unified method to verifying almost all inference-based properties (e.g., detectability, diagnosability, and predictability), and removes two fundamental assumptions of deadlock-freeness and divergence-freeness used for nearly 30 years;
- ConCom + observer provide the most efficient algorithms for verifying concealment-based properties (e.g., all kinds of state-based opacity)[a].

[a] K. Zhang (2023a). "A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems". In: *Annual Reviews in Control* 56, 100902(1–21).

The classical tools — detector[a], twin-plant[b], and verifier[c] all depend on the above two assumptions. The three tools and ConCom have almost the same complexity.

[a] S. Shu and F. Lin (2011). "Generalized detectability for discrete event systems". In: *Systems & Control Letters* 60.5, pp. 310–317.

[b] S. Jiang, Z. Huang, V. Chandra, and R. Kumar (Aug. 2001). "A polynomial algorithm for testing diagnosability of discrete-event systems". In: *IEEE Transactions on Automatic Control* 46.8, pp. 1318–1321.

[c] T.-S. Yoo and S. Lafortune (Sept. 2002). "Polynomial-time verification of diagnosability of partially observed discrete-event systems". In: *IEEE Transactions on Automatic Control* 47.9, pp. 1491–1495; S. Genc and S. Lafortune (2009). "Predictability of event occurrences in partially-observed discrete-event systems". In: *Automatica* 45.2, pp. 301–311.

properties	inference-based	concealment-based
verification methods	concurrent composition (Zhang and Giua, 2020)	concurrent composition +observer (Han et al., 2023)
	currently the most efficient	currently the most efficient
	remove two fundamental assumptions of deadlock-freeness and divergence-freeness used for nearly 30 years	observer (Rabin and Scott, 1959) 1976 Turing Award

Table 1: State-of-the-art in labeled finite-state automata (Zhang, 2023).


Zhang, K. and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.


Han, X., K. Zhang, J. Zhang, Z. Li, and Z. Chen (2023). "Strong current-state and initial-state opacity of discrete-event systems". In: *Automatica* 148, 110756(1–8).


Rabin, M.O. and D. Scott (1959). "Finite automata and their decision problems". In: *IBM Journal of Research and Development* 3.2, pp. 114–125.


Zhang, K. (2023). "A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems". In: *Annual Reviews in Control* 56, 100902(1–21).

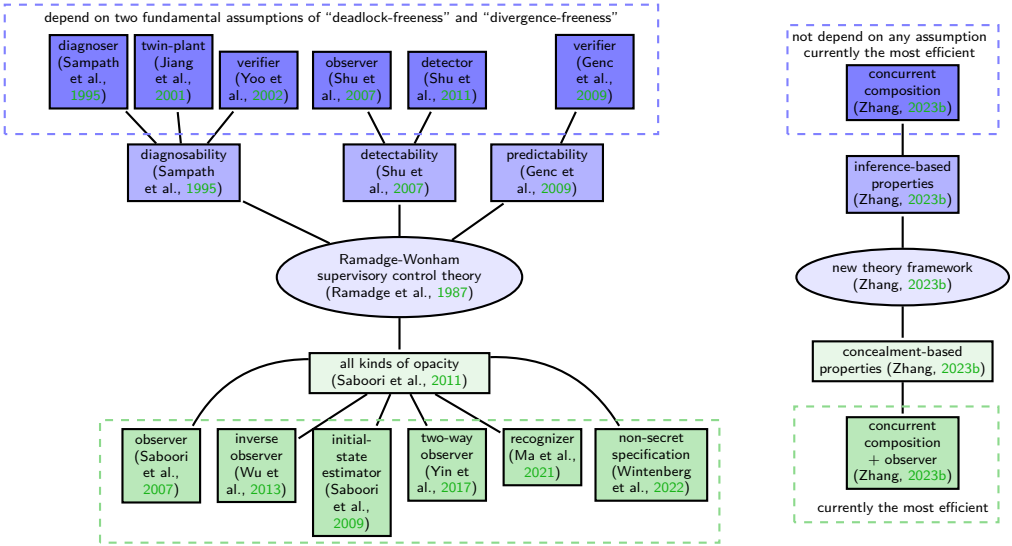


Figure 4: State-of-the-art in discrete-event systems modeled by labeled finite-state automata.

	concurrent composition	observer
labeled finite-state automata	P (Zhang and Giua, 2020)	EXPTIME (Rabin and Scott, 1959) 1976 Turing Award
labeled weighted automata over monoid $(\mathbb{Q}^k, +)$	NP-complete (Zhang, 2022)	N2EXPTIME (Zhang, 2022)
labeled real-time automata	NP-complete (Yan and Zhang, 2024)	N2EXPTIME (Zhang, 2024)

Zhang, K. and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Rabin, M.O. and D. Scott (1959). "Finite automata and their decision problems". In: *IBM Journal of Research and Development* 3.2, pp. 114–125.

Zhang, K. (2022). "Detectability of labeled weighted automata over monoids". In: *Discrete Event Dynamic Systems* 32.3, pp. 435–494.

Yan, Y. and K. Zhang (2024). "Formulation of concurrent composition in labeled real-time automata with an application to fault diagnosis". In: *2024 UKACC 14th International Conference on Control (CONTROL)*, pp. 281–286.

Zhang, K. (2024). "State-based opacity of labeled real-time automata". In: *Theoretical Computer Science* 987, 114373(1–20).

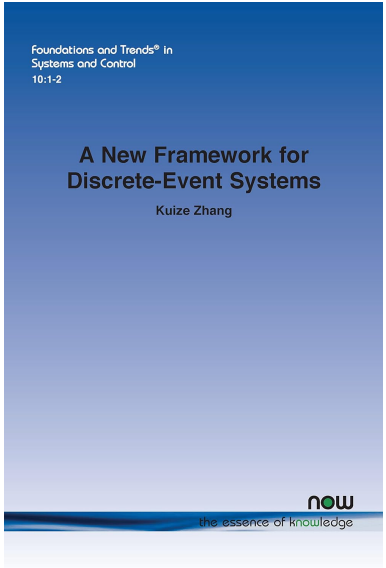
public comments[a] (https://www.nowpublishers.com/article/Details/SYS-028)

[a] K. Zhang (2023b). "A new framework for discrete-event systems". In: *Foundations and Trends® in Systems and Control* 10.1-2, pp. 1–179.

This book introduces the reader to a new paradigm in discrete event dynamic systems. It provides researchers, students and practitioners with the basic theory and a set on implementable tools that will have a significant impact on systems of the future.

ConCom+ observer rewrite the textbook of discrete-event systems.

截止目前，整个中国大陆尚无一部完整的专著出版于 Foundations and Trends in Systems and Control.



A labeled finite-state automaton (LFSA) is a sextuple

$$\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell), \tag{1}$$

where Q is a finite set of states, E a finite alphabet of events, $\delta \subset Q \times E \times Q$ the set of transitions (equivalently represented by transition function $\delta : Q \times E \to 2^Q$), $Q_0 \subset Q$ a set of initial states, Σ a finite alphabet of outputs/labels, $\ell : E \to \Sigma \cup \{\epsilon\}$ the labeling function, ϵ the empty word.

A labeled finite-state automaton (LFSA) is a sextuple

$$\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell), \tag{1}$$

where Q is a finite set of states, E a finite alphabet of events, $\delta \subset Q \times E \times Q$ the set of transitions (equivalently represented by transition function $\delta : Q \times E \to 2^Q$), $Q_0 \subset Q$ a set of initial states, Σ a finite alphabet of outputs/labels, $\ell : E \to \Sigma \cup \{\epsilon\}$ the labeling function, ϵ the empty word.

transition $q \xrightarrow{e} q'$: when the LFSA is in state q and event e occurs, the LFSA transitions to state q'

A labeled finite-state automaton (LFSA) is a sextuple

$$\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell), \tag{1}$$

where Q is a finite set of states, E a finite alphabet of events, $\delta \subset Q \times E \times Q$ the set of transitions (equivalently represented by transition function $\delta : Q \times E \to 2^Q$), $Q_0 \subset Q$ a set of initial states, Σ a finite alphabet of outputs/labels, $\ell : E \to \Sigma \cup \{\epsilon\}$ the labeling function, ϵ the empty word.

transition $q \xrightarrow{e} q'$: when the LFSA is in state q and event e occurs, the LFSA transitions to state q'

A finite-state automaton (FSA)

$$G = (Q, E, \delta, Q_0).$$

Notation

- ℓ is recursively extended to $E^* \cup E^\omega \to \Sigma^* \cup \Sigma^\omega$
- observable events: $E_o = \{e \in E | \ell(e) \in \Sigma\}$
- unobservable events: $E_{uo} = \{e \in E | \ell(e) = \epsilon\}$
- run: $q_0 \xrightarrow{e_1} q_1 \xrightarrow{e_2} \cdots \xrightarrow{e_n} q_n$ (or $q_0 \xrightarrow{e_1 \ldots e_n} q_n$)
- generated language $L(\mathcal{S})$: the set of all such $e_1 \ldots e_n$ and ϵ
- Infinite run: $q_0 \xrightarrow{e_1} q_1 \xrightarrow{e_2} \cdots$
- generated ω-language $L^\omega(\mathcal{S})$: the set of all such $e_1 e_2 \ldots$

current-state estimate

$\mathcal{M}(\mathcal{S}, \alpha)$ with $\alpha \in \Sigma^*$:

$$\{q \in Q | (\exists q_0 \in Q_0)(\exists s \in E^*)[(q_0 \xrightarrow{s} q) \wedge (\ell(s) = \alpha)]\}$$

After $\alpha \in \Sigma^*$ has been generated, the set of states that $\mathcal{S}$ can be in.

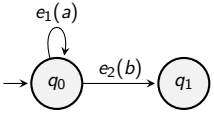
current-state estimate

$\mathcal{M}(\mathcal{S}, \alpha)$ with $\alpha \in \Sigma^*$:

$$\{q \in Q | (\exists q_0 \in Q_0)(\exists s \in E^*)[(q_0 \xrightarrow{s} q) \wedge (\ell(s) = \alpha)]\}$$

After $\alpha \in \Sigma^*$ has been generated, the set of states that $\mathcal{S}$ can be in.

Example 1



LFSA $\mathcal{S}_1$, $L(\mathcal{S}_1) = \{(e_1)^n, (e_1)^n e_2 | n \geq 0\}$, $L^\omega(\mathcal{S}_1) = \{(e_1)^\omega\}$, $\mathcal{M}(\mathcal{S}_1, b) = \{q_1\}$.

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(e_1, e_2) | e_1 \in E_{1o}, e_2 \in E_{2o}, \ell_1(e_1) = \ell_2(e_2)\}$, $E'_{uo} = \{(e_{1uo}, \epsilon) | e_{1uo} \in E_{1uo}\} \cup \{(\epsilon, e_{2uo}) | e_{2uo} \in E_{2uo}\}$;

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(e_1, e_2) | e_1 \in E_{1o}, e_2 \in E_{2o}, \ell_1(e_1) = \ell_2(e_2)\}$, $E'_{uo} = \{(e_{1uo}, \epsilon) | e_{1uo} \in E_{1uo}\} \cup \{(\epsilon, e_{2uo}) | e_{2uo} \in E_{2uo}\}$;
3. $Q'_0 = Q_{01} \times Q_{02}$;

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(e_1, e_2) | e_1 \in E_{1o}, e_2 \in E_{2o}, \ell_1(e_1) = \ell_2(e_2)\}$, $E'_{uo} = \{(e_{1uo}, \epsilon) | e_{1uo} \in E_{1uo}\} \cup \{(\epsilon, e_{2uo}) | e_{2uo} \in E_{2uo}\}$;
3. $Q'_0 = Q_{01} \times Q_{02}$;
4. for all $(q_1, q_3), (q_2, q_4) \in Q'$, $(e_1, e_2) \in E'_o$, $(e_{1uo}, \epsilon), (\epsilon, e_{2uo}) \in E'_{uo}$,

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(e_1, e_2) | e_1 \in E_{1o}, e_2 \in E_{2o}, \ell_1(e_1) = \ell_2(e_2)\}$, $E'_{uo} = \{(e_{1uo}, \epsilon) | e_{1uo} \in E_{1uo}\} \cup \{(\epsilon, e_{2uo}) | e_{2uo} \in E_{2uo}\}$;
3. $Q'_0 = Q_{01} \times Q_{02}$;
4. for all $(q_1, q_3), (q_2, q_4) \in Q'$, $(e_1, e_2) \in E'_o$, $(e_{1uo}, \epsilon), (\epsilon, e_{2uo}) \in E'_{uo}$,
   - $((q_1, q_3), (e_1, e_2), (q_2, q_4)) \in \delta'$ iff $(q_1, e_1, q_2) \in \delta_1, (q_3, e_2, q_4) \in \delta_2$,

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(e_1, e_2) | e_1 \in E_{1o}, e_2 \in E_{2o}, \ell_1(e_1) = \ell_2(e_2)\}$, $E'_{uo} = \{(e_{1uo}, \epsilon) | e_{1uo} \in E_{1uo}\} \cup \{(\epsilon, e_{2uo}) | e_{2uo} \in E_{2uo}\}$;
3. $Q'_0 = Q_{01} \times Q_{02}$;
4. for all $(q_1, q_3), (q_2, q_4) \in Q'$, $(e_1, e_2) \in E'_o$, $(e_{1uo}, \epsilon), (\epsilon, e_{2uo}) \in E'_{uo}$,
   - $((q_1, q_3), (e_1, e_2), (q_2, q_4)) \in \delta'$ iff $(q_1, e_1, q_2) \in \delta_1, (q_3, e_2, q_4) \in \delta_2$,
   - $((q_1, q_3), (e_{1uo}, \epsilon), (q_2, q_4)) \in \delta'$ iff $(q_1, e_{1uo}, q_2) \in \delta_1$, $q_3 = q_4$,

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(e_1, e_2) | e_1 \in E_{1o}, e_2 \in E_{2o}, \ell_1(e_1) = \ell_2(e_2)\}$, $E'_{uo} = \{(e_{1uo}, \epsilon) | e_{1uo} \in E_{1uo}\} \cup \{(\epsilon, e_{2uo}) | e_{2uo} \in E_{2uo}\}$;
3. $Q'_0 = Q_{01} \times Q_{02}$;
4. for all $(q_1, q_3), (q_2, q_4) \in Q'$, $(e_1, e_2) \in E'_o$, $(e_{1uo}, \epsilon), (\epsilon, e_{2uo}) \in E'_{uo}$,
   - $((q_1, q_3), (e_1, e_2), (q_2, q_4)) \in \delta'$ iff $(q_1, e_1, q_2) \in \delta_1, (q_3, e_2, q_4) \in \delta_2$,
   - $((q_1, q_3), (e_{1uo}, \epsilon), (q_2, q_4)) \in \delta'$ iff $(q_1, e_{1uo}, q_2) \in \delta_1$, $q_3 = q_4$,
   - $((q_1, q_3), (\epsilon, e_{2uo}), (q_2, q_4)) \in \delta'$ iff $q_1 = q_2$, $(q_3, e_{2uo}, q_4) \in \delta_2$,

Definition 7 (Concurrent composition[a])

[a] K. Zhang and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Consider two LFSAs $\mathcal{S}^i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i)$, $i = 1, 2$, define the concurrent composition (ConCom) $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ of $\mathcal{S}^1$ and $\mathcal{S}^2$ by

$$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2) = (Q', E', \delta', Q'_0, \Sigma, \ell'), \tag{2}$$

1. $Q' = Q_1 \times Q_2$;
2. $E' = E'_o \cup E'_{uo}$, where $E'_o = \{(e_1, e_2) | e_1 \in E_{1o}, e_2 \in E_{2o}, \ell_1(e_1) = \ell_2(e_2)\}$, $E'_{uo} = \{(e_{1uo}, \epsilon) | e_{1uo} \in E_{1uo}\} \cup \{(\epsilon, e_{2uo}) | e_{2uo} \in E_{2uo}\}$;
3. $Q'_0 = Q_{01} \times Q_{02}$;
4. for all $(q_1, q_3), (q_2, q_4) \in Q'$, $(e_1, e_2) \in E'_o$, $(e_{1uo}, \epsilon), (\epsilon, e_{2uo}) \in E'_{uo}$,
   - $((q_1, q_3), (e_1, e_2), (q_2, q_4)) \in \delta'$ iff $(q_1, e_1, q_2) \in \delta_1, (q_3, e_2, q_4) \in \delta_2$,
   - $((q_1, q_3), (e_{1uo}, \epsilon), (q_2, q_4)) \in \delta'$ iff $(q_1, e_{1uo}, q_2) \in \delta_1$, $q_3 = q_4$,
   - $((q_1, q_3), (\epsilon, e_{2uo}), (q_2, q_4)) \in \delta'$ iff $q_1 = q_2$, $(q_3, e_{2uo}, q_4) \in \delta_2$,
5. for all $(e_1, e_2) \in E'$, $\ell((e_1, e_2)) = \ell_1(e_1) = \ell_2(e_2)$.

Features of $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$

$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$, observable transitions of $\mathcal{S}^1$ and $\mathcal{S}^2$ with the same label are synchronized, but unobservable transitions interleave, so $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ aggregates every pair of runs that produce the same label sequence.

Features of $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$

$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$, observable transitions of $\mathcal{S}^1$ and $\mathcal{S}^2$ with the same label are synchronized, but unobservable transitions interleave, so $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ aggregates every pair of runs that produce the same label sequence.

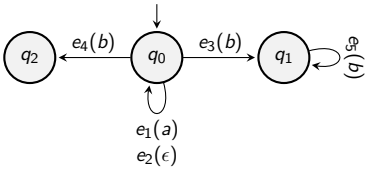
$\mathrm{CC}(\mathcal{S}, \mathcal{S}) =: \mathrm{CC}(\mathcal{S})$ is called the self-composition of $\mathcal{S}$.

Features of $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$

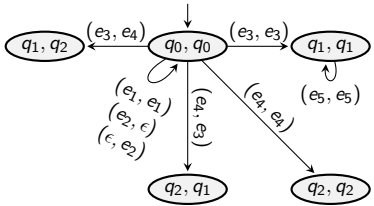
$\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$, observable transitions of $\mathcal{S}^1$ and $\mathcal{S}^2$ with the same label are synchronized, but unobservable transitions interleave, so $\mathrm{CC}(\mathcal{S}^1, \mathcal{S}^2)$ aggregates every pair of runs that produce the same label sequence.

$\mathrm{CC}(\mathcal{S}, \mathcal{S}) =: \mathrm{CC}(\mathcal{S})$ is called the self-composition of $\mathcal{S}$.

Example 8



$\mathcal{S}_2$.



Reachable part of $\mathrm{CC}(\mathcal{S}_2)$.

Definition 9 (powerset construction, Rabin and Scott, 1959)

Consider an LFSA $\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell)$. Its observer $\mathcal{S}_{\text{obs}}$[a] is defined by a deterministic finite automaton

$$(Q_{\text{obs}}, \ell(E_o), \delta_{\text{obs}}, q_{0\,\text{obs}}),$$

where

1. $Q_{\text{obs}} = 2^Q$,
2. $\ell(E_o) = \ell(E) \setminus \{\epsilon\}$,
3. for all $x \in Q_{\text{obs}}$ and $a \in \ell(E_o)$, $\delta_{\text{obs}}(x, a) = \bigcup_{q \in x} \bigcup_{\substack{e_a \in E_o \\ \ell(e_a) = a \\ s \in (E_{uo})^*}} \delta(q, e_a s)$,
4. $q_{0\,\text{obs}} = \bigcup_{q_0 \in Q_0} \bigcup_{s \in (E_{uo})^*} \delta(q_0, s)$ (i.e., $\mathcal{M}(\mathcal{S}, \epsilon)$).

[a] The term "observer" dates back to (Özveren and Willsky, 1990; Shu, Lin, and Ying, 2007).

Definition 9 (powerset construction, Rabin and Scott, 1959)

Consider an LFSA $\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell)$. Its observer $\mathcal{S}_{\text{obs}}$[a] is defined by a deterministic finite automaton

$$(Q_{\text{obs}}, \ell(E_o), \delta_{\text{obs}}, q_{0\,\text{obs}}),$$

where

1. $Q_{\text{obs}} = 2^Q$,
2. $\ell(E_o) = \ell(E) \setminus \{\epsilon\}$,
3. for all $x \in Q_{\text{obs}}$ and $a \in \ell(E_o)$, $\delta_{\text{obs}}(x, a) = \bigcup_{q \in x} \bigcup_{\substack{e_a \in E_o \\ \ell(e_a) = a \\ s \in (E_{uo})^*}} \delta(q, e_a s)$,
4. $q_{0\,\text{obs}} = \bigcup_{q_0 \in Q_0} \bigcup_{s \in (E_{uo})^*} \delta(q_0, s)$ (i.e., $\mathcal{M}(\mathcal{S}, \epsilon)$).

[a] The term "observer" dates back to (Özveren and Willsky, 1990; Shu, Lin, and Ying, 2007).

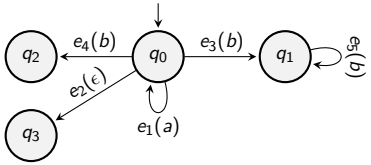
Features of $\mathcal{S}_{\text{obs}}$

$\mathcal{S}_{\text{obs}}$ collects the state estimates of $\mathcal{S}$ wrt all generated label sequences.

For every $\alpha \in \Sigma^*$, $\delta_{\mathsf{obs}}(q_{0\,\mathsf{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

For every $\alpha \in \Sigma^*$, $\delta_{\text{obs}}(q_{0\,\text{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

Example 10

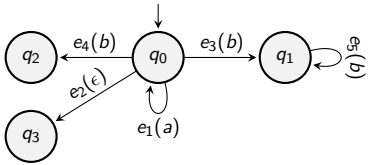


LFSA $\mathcal{S}_3$.

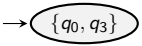
Part of observer $\mathcal{S}_{3\,\text{obs}}$ of $\mathcal{S}_3$.

For every $\alpha \in \Sigma^*$, $\delta_{\text{obs}}(q_{0\,\text{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

Example 10



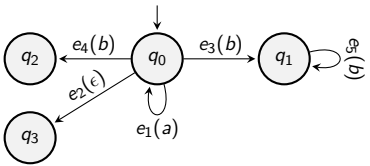
LFSA $\mathcal{S}_3$.



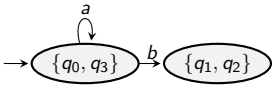
Part of observer $\mathcal{S}_{3\,\text{obs}}$ of $\mathcal{S}_3$.

For every $\alpha \in \Sigma^*$, $\delta_{\text{obs}}(q_{0\,\text{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

Example 10



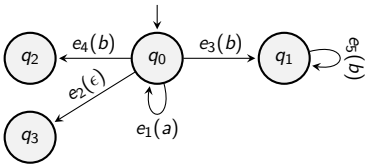
LFSA $\mathcal{S}_3$.



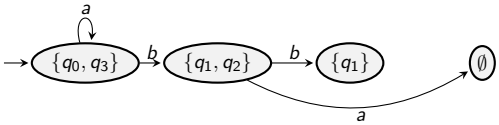
Part of observer $\mathcal{S}_{3\,\text{obs}}$ of $\mathcal{S}_3$.

For every $\alpha \in \Sigma^*$, $\delta_{\text{obs}}(q_{0\,\text{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

Example 10



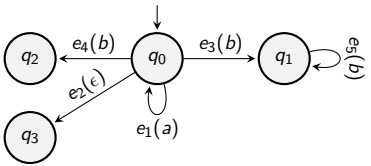
LFSA $\mathcal{S}_3$.



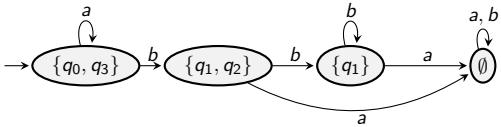
Part of observer $\mathcal{S}_{3\,\text{obs}}$ of $\mathcal{S}_3$.

For every $\alpha \in \Sigma^*$, $\delta_{\text{obs}}(q_{0\,\text{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

Example 10



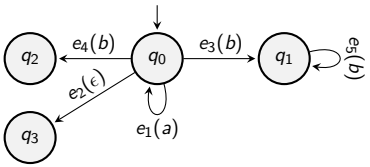
LFSA $\mathcal{S}_3$.



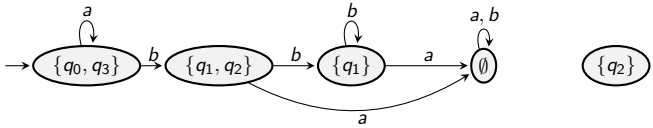
Part of observer $\mathcal{S}_{3\,\text{obs}}$ of $\mathcal{S}_3$.

For every $\alpha \in \Sigma^*$, $\delta_{\text{obs}}(q_{0\,\text{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

Example 10



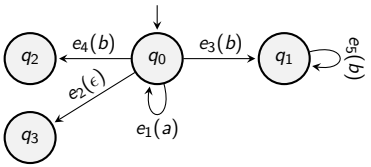
LFSA $\mathcal{S}_3$.



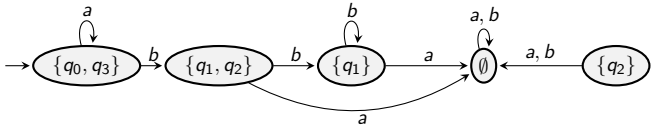
Part of observer $\mathcal{S}_{3\,\text{obs}}$ of $\mathcal{S}_3$.

For every $\alpha \in \Sigma^*$, $\delta_{\text{obs}}(q_{0\,\text{obs}}, \alpha) = \mathcal{M}(\mathcal{S}, \alpha)$.

Example 10



LFSA $\mathcal{S}_3$.



Part of observer $\mathcal{S}_{3\,\text{obs}}$ of $\mathcal{S}_3$.

Content

1. Motivation
2. Review of basic tools — concurrent composition and observer
3. A system consisting of a single agent
   - A general framework
   - Case studies
4. A system consisting of two ordered agents
   - A general framework
   - Case studies
5. A system consisting of a finite number of ordered agents
   - A general framework
   - Case studies
6. Conclusion

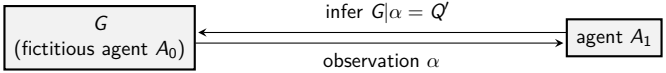


Figure 6: Order-1 property. Based on the observation α of A_1 to G, A_1 infers the set Q' of states G with respect to α.

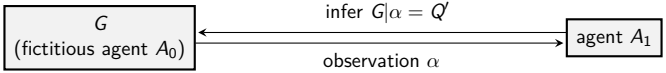


Figure 6: Order-1 property. Based on the observation α of A_1 to G, A_1 infers the set Q' of states G with respect to α.

Definition 11 (order-1 predicate)

$$\mathrm{PRED}^1 \subset 2^Q. \tag{3}$$

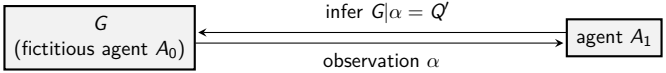


Figure 6: Order-1 property. Based on the observation α of A_1 to G, A_1 infers the set Q' of states G with respect to α.

Definition 11 (order-1 predicate)

$$\mathrm{PRED}^1 \subset 2^Q. \tag{3}$$

The setting

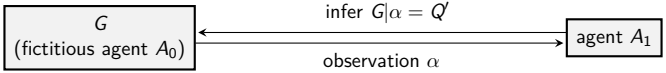


Figure 6: Order-1 property. Based on the observation α of A_1 to G, A_1 infers the set Q' of states G with respect to α.

Definition 11 (order-1 predicate)

$$\mathrm{PRED}^1 \subset 2^Q. \tag{3}$$

The setting

- Agent A_1 knows $G = (Q, E, \delta, Q_0)$.

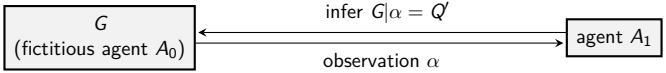


Figure 6: Order-1 property. Based on the observation α of A_1 to G, A_1 infers the set Q' of states G with respect to α.

Definition 11 (order-1 predicate)

$$\mathrm{PRED}^1 \subset 2^Q. \tag{3}$$

The setting

- Agent A_1 knows $G = (Q, E, \delta, Q_0)$.
- A_1 observes $E_1 \subset E$ via labeling function ℓ_1: $E_1 \to \Sigma_1$, $E \setminus E_1 \to \{\epsilon\}$.

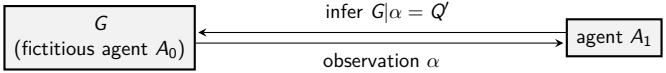


Figure 6: Order-1 property. Based on the observation α of A_1 to G, A_1 infers the set Q' of states G with respect to α.

Definition 11 (order-1 predicate)

$$\mathrm{PRED}^1 \subset 2^Q. \tag{3}$$

The setting

- Agent A_1 knows $G = (Q, E, \delta, Q_0)$.
- A_1 observes $E_1 \subset E$ via labeling function ℓ_1: $E_1 \to \Sigma_1$, $E \setminus E_1 \to \{\epsilon\}$.
- Current-state estimate $\mathcal{M}((G, \Sigma_1, \ell_1), \alpha) =: \mathcal{M}_1(G, \alpha)$ of A_1 with respect to observation $\alpha \in (\Sigma_1)^* \cap \ell_1(L(G))$ to G.

Definition 12

An FSA G satisfies the order-1 estimation-based property PRED^1 with respect to agent A_1 if

$$\{\mathcal{M}_1(G, \alpha) | \alpha \in \ell_1(L(G))\} \subset \text{PRED}^1. \tag{4}$$

Definition 12

An FSA G satisfies the order-1 estimation-based property PRED^1 with respect to agent A_1 if

$$\{\mathcal{M}_1(G,\alpha)|\alpha \in \ell_1(L(G))\} \subset \text{PRED}^1. \tag{4}$$

Theorem 13

An FSA G satisfies the order-1 estimation-based property PRED^1 with respect to agent A_1 if and only if in the observer $\text{Obs}_1(G)$, every reachable state X belongs to PRED^1.

Examples of order-1 estimation-based properties

static ($\mathrm{Obs}(\mathcal{S})$)	dynamic ($\mathrm{Obs}(\mathrm{CC}(\mathcal{S}, \bar{\mathcal{S}}))$)
current-state opacity[1][2] $\{X \subset Q \mid X \not\subset Q_S\}$	regular-language-based opacity[3] $\{X \subset Q_1 \times Q_2 \mid$ $(\exists q_1 \in F_1)(\exists q_2 \in Q_2)[(q_1, q_2) \in X]$ $\Longrightarrow\ (\exists q_3 \in F_2)[(q_1, q_3) \in X]\}$
critical observability[4] $\{X \subset Q \mid X \subset Q_{\mathrm{CriObs}} \text{ or } X \cap Q_{\mathrm{CriObs}} = \emptyset\}$	strong current-state opacity[5] $\{X \subset Q \times Q_{\mathrm{NS}}^{\diamond} \mid$ $(\exists q_1 \in Q_S)(\exists q_2 \in Q_{\mathrm{NS}}^{\diamond})[(q_1, q_2) \in X]$ $\Longrightarrow\ (\exists q_3 \in Q_{\mathrm{NS}})[(q_1, q_3) \in X]\}$
$\vdots$	$\vdots$

[1] A. Saboori and C.N. Hadjicostis (Dec. 2007). "Notions of security and opacity in discrete event systems". In: *46th IEEE Conference on Decision and Control*, pp. 5056–5061.

[2] F. Cassez, J. Dubreil, and H. Marchand (2009). "Dynamic observers for the synthesis of opaque systems". In: *Automated Technology for Verification and Analysis*. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 352–367.

[3] F. Lin (2011). "Opacity of discrete event systems and its applications". In: *Automatica* 47.3, pp. 496–503.

[4] G. Pola, E. De Santis, M.D. Di Benedetto, and D. Pezzuti (2017). "Design of decentralized critical observers for networks of finite state machines: a formal method approach". In: *Automatica* 86, pp. 174–182.

[5] X. Han et al. (2023). "Strong current-state and initial-state opacity of discrete-event systems". In: *Automatica* 148, 110756(1–8).

Case study 1: Current-state opacity (static)

Specify a subset $Q_S \subset Q$ of secret states.

Case study 1: Current-state opacity (static)

Specify a subset $Q_S \subset Q$ of secret states.

Definition 14 (Saboori and Hadjicostis, 2007)

Consider an LFSA (G, Σ_1, ℓ_1) and a subset $Q_S \subset Q$ of secret states. FSA G is called current-state opaque with respect to ℓ_1 and Q_S if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0$ and $q \in Q_S$, there exists a run $q_0' \xrightarrow{s'} q'$ such that $q_0' \in Q_0$, $q' \in Q \setminus Q_S$, and $\ell_1(s) = \ell_1(s')$.

Current-state opacity (Cassez, Dubreil, and Marchand, 2009; Saboori and Hadjicostis, 2007) means that, whenever a secret state is visited at the end of a run, there is another run whose final state is not secret such that the two runs look the same to an intruder.


Saboori, A. and C.N. Hadjicostis (Dec. 2007). "Notions of security and opacity in discrete event systems". In: *46th IEEE Conference on Decision and Control*, pp. 5056–5061.


Cassez, F., J. Dubreil, and H. Marchand (2009). "Dynamic observers for the synthesis of opaque systems". In: *Automated Technology for Verification and Analysis*. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 352–367.

Case study 1: Current-state opacity (static)

$$\mathrm{PRED}^1(G, Q_S) := \{X \subset Q | X \not\subset Q_S\} \subset 2^Q. \tag{5}$$

Case study 1: Current-state opacity (static)

$$\mathrm{PRED}^1(G, Q_S) := \{X \subset Q | X \not\subset Q_S\} \subset 2^Q. \tag{5}$$

Definition 15

An FSA G is called current-state opaque with respect to ℓ_1 and Q_S if

$$\{\mathcal{M}_1(G, \alpha) | \alpha \in \ell_1(L(G))\} \subset \mathrm{PRED}^1(G, Q_S).$$

Case study 1: Current-state opacity (static)

$$\mathrm{PRED}^1(G, Q_S) := \{X \subset Q | X \not\subset Q_S\} \subset 2^Q. \tag{5}$$

Definition 15

An FSA G is called current-state opaque with respect to ℓ_1 and Q_S if

$$\{\mathcal{M}_1(G, \alpha) | \alpha \in \ell_1(L(G))\} \subset \mathrm{PRED}^1(G, Q_S).$$

Theorem 16 (Cassez, Dubreil, and Marchand, 2009)

The current-state opacity verification problem is PSPACE*-complete in LFSAs.*

Case study 2: A generalized version of critical observability (static)

Definition 17

An FSA G is called critically observable with respect to ℓ_1 and partition $\{Q^1_{\text{CriObs}}, \ldots, Q^n_{\text{CriObs}}\}$[a] of Q if for every $\alpha \in \ell_1(L(G))$, $\mathcal{M}_1(G, \alpha) \subset Q^i_{\text{CriObs}}$ for exactly one i in $[\![1, n]\!]$.

[a] That is, $Q^1_{\text{CriObs}}, \ldots, Q^n_{\text{CriObs}}$ are nonempty, pairwise disjoint, and satisfy $\bigcup_{i=1}^n Q^i_{\text{CriObs}} = Q$.

Case study 2: A generalized version of critical observability (static)

Definition 17

An FSA G is called critically observable with respect to ℓ_1 and partition $\{Q^1_{\text{CriObs}}, \ldots, Q^n_{\text{CriObs}}\}$[a] of Q if for every $\alpha \in \ell_1(L(G))$, $\mathcal{M}_1(G, \alpha) \subset Q^i_{\text{CriObs}}$ for exactly one i in $[\![1, n]\!]$.

[a] That is, $Q^1_{\text{CriObs}}, \ldots, Q^n_{\text{CriObs}}$ are nonempty, pairwise disjoint, and satisfy $\bigcup_{i=1}^n Q^i_{\text{CriObs}} = Q$.

$$\text{PRED}^1_{\text{CriObs}} := \{X \subset Q | X \subset Q^i_{\text{CriObs}} \text{ for exactly one } i \in [\![1, n]\!]\} \subset 2^Q. \tag{6}$$

Case study 2: A generalized version of critical observability (static)

Definition 17

An FSA G is called critically observable with respect to ℓ_1 and partition $\{Q^1_{\text{CriObs}}, \ldots, Q^n_{\text{CriObs}}\}$[a] of Q if for every $\alpha \in \ell_1(L(G))$, $\mathcal{M}_1(G, \alpha) \subset Q^i_{\text{CriObs}}$ for exactly one i in $[\![1, n]\!]$.

[a] That is, $Q^1_{\text{CriObs}}, \ldots, Q^n_{\text{CriObs}}$ are nonempty, pairwise disjoint, and satisfy $\bigcup_{i=1}^n Q^i_{\text{CriObs}} = Q$.

$$\text{PRED}^1_{\text{CriObs}} := \{X \subset Q | X \subset Q^i_{\text{CriObs}} \text{ for exactly one } i \in [\![1, n]\!]\} \subset 2^Q. \tag{6}$$

Definition 18

An FSA G is called critically observable with respect to ℓ_1 and $\{Q^1_{\text{CriObs}}, \ldots, Q^n_{\text{CriObs}}\}$ if

$$\{\mathcal{M}_1(G, \alpha) | \alpha \in \ell_1(L(G))\} \subset \text{PRED}^1_{\text{CriObs}}.$$

Case study 3: Regular-language-based opacity (dynamic)

Definition 19 (Lin, 2011)

A regular language L_1 is called L-opaque with respect to a regular language L_2 and labeling functions ℓ_1, ℓ_2 if $\ell_1(L_1) \subset \ell_2(L_2)$.

Case study 3: Regular-language-based opacity (dynamic)

Definition 19 (Lin, 2011)

A regular language L_1 is called L-opaque with respect to a regular language L_2 and labeling functions ℓ_1, ℓ_2 if $\ell_1(L_1) \subset \ell_2(L_2)$.

Without loss of generality, assume

for every run $q_{01} \xrightarrow{s_1} q_1$ in G_1, there is a run $q_{02} \xrightarrow{s_2} q_2$ in G_2 such that $\ell_1(s_1) = \ell_2(s_2)$, where $q_{0i} \in Q_{0i}$, $i = 1, 2$. (D)

Case study 3: Regular-language-based opacity (dynamic)

Definition 19 (Lin, 2011)

A regular language L_1 is called L-opaque with respect to a regular language L_2 and labeling functions ℓ_1, ℓ_2 if $\ell_1(L_1) \subset \ell_2(L_2)$.

Without loss of generality, assume

for every run $q_{01} \xrightarrow{s_1} q_1$ in G_1, there is a run $q_{02} \xrightarrow{s_2} q_2$ in G_2 such that $\ell_1(s_1) = \ell_2(s_2)$, where $q_{0i} \in Q_{0i}$, $i = 1, 2$. (D)

Theorem 20

Consider two LFSAs $\mathcal{S}_i = (G_i, \Sigma_i, \ell_i)$, $i = 1, 2$, where $G_i = (Q_i, \Sigma_i, \delta_i, Q_{0i}, F_i)$ is an NFA and recognizes regular language L_i. Without loss of generality, assume G_1 and G_2 satisfy (D). *Regular language L_1 is L-opaque with respect to regular language L_2 and labeling functions ℓ_1, ℓ_2 if and only if in* $\mathrm{Obs}(\mathrm{CC}(\mathcal{S}_1, \mathcal{S}_2))$, *in each reachable state X of* $\mathrm{Obs}(\mathrm{CC}(\mathcal{S}_1, \mathcal{S}_2))$, *if there is a pair (q_1, q_2) such that $q_1 \in F_1$, then there is another pair (q_1, q_3) such that $q_3 \in F_2$.*

Case study 3: Regular-language-based opacity (dynamic)

$$
\begin{aligned}
&\mathrm{PRED}^1(Q_1, Q_2, F_1, F_2) \\
:=&\{X \subset Q_1 \times Q_2 | (\exists q_1 \in F_1)(\exists q_2 \in Q_2)[(q_1, q_2) \in X] \\
&\qquad \Longrightarrow (\exists q_3 \in F_2)[(q_1, q_3) \in X]\} \\
\subset& 2^{Q_1 \times Q_2}.
\end{aligned} \tag{7}
$$

Case study 3: Regular-language-based opacity (dynamic)

$$\begin{aligned}&\mathrm{PRED}^1(Q_1, Q_2, F_1, F_2)\\ :=&\{X \subset Q_1 \times Q_2 | (\exists q_1 \in F_1)(\exists q_2 \in Q_2)[(q_1, q_2) \in X]\\ &\qquad\qquad \Longrightarrow (\exists q_3 \in F_2)[(q_1, q_3) \in X]\}\\ \subset& 2^{Q_1 \times Q_2}.\end{aligned} \tag{7}$$

Definition 21

A regular language L_1 is called L-opaque with respect to a regular language L_2 and labeling functions ℓ_1, ℓ_2 if

$$(Q_1 \times Q_2)_{\mathrm{Obs}} \subset \mathrm{PRED}^1(Q_1, Q_2, F_1, F_2), \tag{8}$$

where for $i = 1, 2$, NFA $G_i = (Q_i, \Sigma_i, \delta_i, Q_{0i}, F_i)$ recognizes language L_i, $\mathcal{S}_i = (G_i, \Sigma, \ell_i)$, G_1 and G_2 satisfy (D). In addition, $(Q_1 \times Q_2)_{\mathrm{Obs}}$ denotes the set of reachable states of observer $\mathrm{Obs}(\mathrm{CC}(\mathcal{S}_1, \mathcal{S}_2))$.

Case study 4: Strong current-state opacity (dynamic)

Definition 22 (Han et al., 2023)

Consider an LFSA $(G, \Sigma_1, \ell_1) =: \mathcal{S}$ and a subset $Q_S \subset Q$ of secret states. FSA G is called strongly current-state opaque with respect to ℓ_1 and Q_S, if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0$ and $q \in Q_S$, there exists a non-secret run $q'_0 \xrightarrow{s'} q'$ such that $q'_0 \in Q_0$ and $\ell(s) = \ell(s')$.

Han, X., K. Zhang, J. Zhang, Z. Li, and Z. Chen (2023). "Strong current-state and initial-state opacity of discrete-event systems". In: *Automatica* 148, 110756(1–8).

Case study 4: Strong current-state opacity (dynamic)

Definition 22 (Han et al., 2023)

Consider an LFSA $(G, \Sigma_1, \ell_1) =: \mathcal{S}$ and a subset $Q_S \subset Q$ of secret states. FSA G is called strongly current-state opaque with respect to ℓ_1 and Q_S, if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0$ and $q \in Q_S$, there exists a non-secret run $q_0' \xrightarrow{s'} q'$ such that $q_0' \in Q_0$ and $\ell(s) = \ell(s')$.

Strong current-state opacity can be reduced to regular-language-based opacity.


Han, X., K. Zhang, J. Zhang, Z. Li, and Z. Chen (2023). "Strong current-state and initial-state opacity of discrete-event systems". In: *Automatica* 148, 110756(1–8).

Case study 4: Strong current-state opacity (dynamic)

Definition 22 (Han et al., 2023)

Consider an LFSA $(G, \Sigma_1, \ell_1) =: \mathcal{S}$ and a subset $Q_S \subset Q$ of secret states. FSA G is called strongly current-state opaque with respect to ℓ_1 and Q_S, if for every run $q_0 \xrightarrow{s} q$ with $q_0 \in Q_0$ and $q \in Q_S$, there exists a non-secret run $q_0' \xrightarrow{s'} q'$ such that $q_0' \in Q_0$ and $\ell(s) = \ell(s')$.

Strong current-state opacity can be reduced to regular-language-based opacity.

Remark 1

The currently most efficient algorithm for verifying strong current-state opacity is based on $\mathrm{CC}(\mathcal{S}, \mathcal{S}_{\mathrm{dss\,obs}})$ *(Han et al., 2023).*

Han, X., K. Zhang, J. Zhang, Z. Li, and Z. Chen (2023). "Strong current-state and initial-state opacity of discrete-event systems". In: *Automatica* 148, 110756(1–8).

Content

1. Motivation
2. Review of basic tools — concurrent composition and observer
3. A system consisting of a single agent
   - A general framework
   - Case studies
4. A system consisting of two ordered agents
   - A general framework
   - Case studies
5. A system consisting of a finite number of ordered agents
   - A general framework
   - Case studies
6. Conclusion

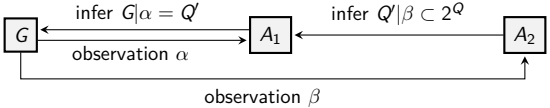


Figure 7: Order-2 property. Given an event sequence $s \in L(G)$, based on the observation $\alpha = \ell_1(s)$ of A_1 to G, A_1 infers the set Q' of all consistent states of G, and then based on the observation $\beta = \ell_2(s)$ of A_2 to G, A_2 infers Q' and obtains a subset of 2^Q.

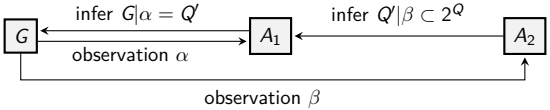


Figure 7: Order-2 property. Given an event sequence $s \in L(G)$, based on the observation $\alpha = \ell_1(s)$ of A_1 to G, A_1 infers the set Q' of all consistent states of G, and then based on the observation $\beta = \ell_2(s)$ of A_2 to G, A_2 infers Q' and obtains a subset of 2^Q.

The setting

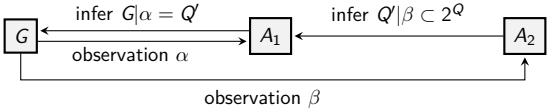


Figure 7: Order-2 property. Given an event sequence $s \in L(G)$, based on the observation $\alpha = \ell_1(s)$ of A_1 to G, A_1 infers the set Q' of all consistent states of G, and then based on the observation $\beta = \ell_2(s)$ of A_2 to G, A_2 infers Q' and obtains a subset of 2^Q.

The setting

- G is known to A_1 and A_2.

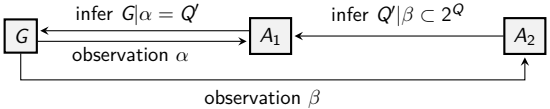


Figure 7: Order-2 property. Given an event sequence $s \in L(G)$, based on the observation $\alpha = \ell_1(s)$ of A_1 to G, A_1 infers the set Q' of all consistent states of G, and then based on the observation $\beta = \ell_2(s)$ of A_2 to G, A_2 infers Q' and obtains a subset of 2^Q.

The setting

- G is known to A_1 and A_2.
- A_i observes $E_i \subset E$ via labeling function $\ell_i : E_i \to \Sigma_i$, $E \setminus E_i \to \{\epsilon\}$, $i = 1, 2$.

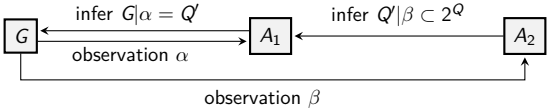


Figure 7: Order-2 property. Given an event sequence $s \in L(G)$, based on the observation $\alpha = \ell_1(s)$ of A_1 to G, A_1 infers the set Q' of all consistent states of G, and then based on the observation $\beta = \ell_2(s)$ of A_2 to G, A_2 infers Q' and obtains a subset of 2^Q.

The setting

- G is known to A_1 and A_2.
- A_i observes $E_i \subset E$ via labeling function $\ell_i : E_i \to \Sigma_i$, $E \setminus E_i \to \{\epsilon\}$, $i = 1, 2$.
- A_2 knows E_1 and ℓ_1.

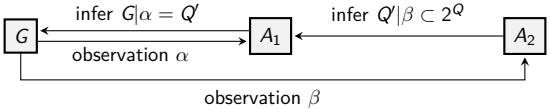


Figure 7: Order-2 property. Given an event sequence $s \in L(G)$, based on the observation $\alpha = \ell_1(s)$ of A_1 to G, A_1 infers the set Q' of all consistent states of G, and then based on the observation $\beta = \ell_2(s)$ of A_2 to G, A_2 infers Q' and obtains a subset of 2^Q.

The setting

- G is known to A_1 and A_2.
- A_i observes $E_i \subset E$ via labeling function $\ell_i : E_i \to \Sigma_i$, $E \setminus E_i \to \{\epsilon\}$, $i = 1, 2$.
- A_2 knows E_1 and ℓ_1.

A member of the office of academic affairs in a university know the scores of all students of the university, a student knows this former situation but only knows his/her own scores. Here the member can be regarded as A_1, each student can be regarded as A_2.

Definition 23 (order-2 predicate)

$$\mathrm{PRED}^2 \subset 2^{2^Q}. \tag{9}$$

Definition 23 (order-2 predicate)

$$\mathrm{PRED}^2 \subset 2^{2^Q}. \tag{9}$$

When A_2 observes $\alpha \in \ell_2(L(G))$, all possible inferences of A_1's current-state estimate of G by A_2 are represented by the set

$$\begin{aligned} &\mathcal{M}_{A_1 \leftarrow A_2}(G, \alpha) \\ :=&\{\mathcal{M}_1(G, \ell_1(s)) | s \in \ell_2^{-1}(\alpha) \cap L(G)\} \subset 2^Q. \end{aligned} \tag{10}$$

Definition 23 (order-2 predicate)

$$\text{PRED}^2 \subset 2^{2^Q}. \tag{9}$$

When A_2 observes $\alpha \in \ell_2(L(G))$, all possible inferences of A_1's current-state estimate of G by A_2 are represented by the set

$$\begin{aligned} &\mathcal{M}_{A_1 \leftarrow A_2}(G, \alpha) \\ :=&\{\mathcal{M}_1(G, \ell_1(s)) | s \in \ell_2^{-1}(\alpha) \cap L(G)\} \subset 2^Q. \end{aligned} \tag{10}$$

Definition 24

An FSA G satisfies the order-2 estimation-based property PRED^2 with respect to agents A_1 and A_2 if

$$\{\mathcal{M}_{A_1 \leftarrow A_2}(G, \alpha) | \alpha \in \ell_2(L(G))\} \subset \text{PRED}^2. \tag{11}$$

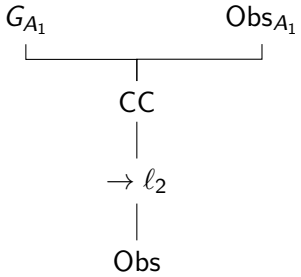


Figure 8: Sketch of the order-2 observer and 2-EXPTIME verification structure for the order-2 estimation-based property.

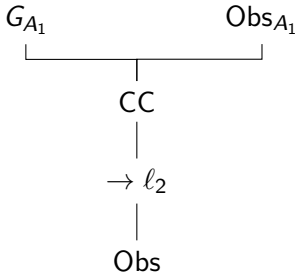


Figure 8: Sketch of the order-2 observer and 2-EXPTIME verification structure for the order-2 estimation-based property.

1. $\mathrm{Obs}_{\ell_1}(G) =: \mathrm{Obs}_{A_1}$.

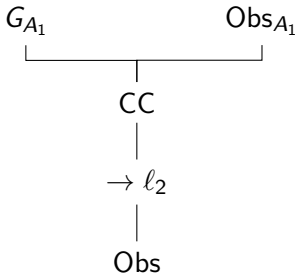


Figure 8: Sketch of the order-2 observer and 2-EXPTIME verification structure for the order-2 estimation-based property.

1. $\mathrm{Obs}_{\ell_1}(G) =: \mathrm{Obs}_{A_1}$.
2. $\mathrm{CC}_{\ell_1}(G_{A_1}, \mathrm{Obs}_{A_1})$. $(e_1, e_2) \to e_1$, $\ell_1 \to \ell_2$, $\mathrm{CC}_{\ell_1}(G_{A_1}, \mathrm{Obs}_{A_1}) \to \mathrm{CC}^{G,\mathrm{Obs}}_{A_1 \to A_2}$.

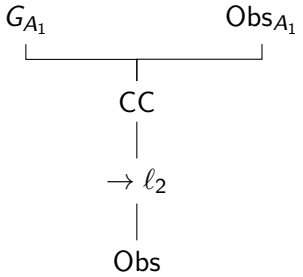


Figure 8: Sketch of the order-2 observer and 2-EXPTIME verification structure for the order-2 estimation-based property.

1. $\mathrm{Obs}_{\ell_1}(G) =: \mathrm{Obs}_{A_1}$.
2. $\mathrm{CC}_{\ell_1}(G_{A_1}, \mathrm{Obs}_{A_1})$. $(e_1, e_2) \to e_1$, $\ell_1 \to \ell_2$, $\mathrm{CC}_{\ell_1}(G_{A_1}, \mathrm{Obs}_{A_1}) \to \mathrm{CC}^{G,\mathrm{Obs}}_{A_1 \to A_2}$.
3. $\mathrm{Obs}_{\ell_2}(\mathrm{CC}^{G,\mathrm{Obs}}_{A_1 \to A_2}) =: \mathrm{Obs}_{A_1 \leftarrow A_2}(G)$ (order-2 observer).

Theorem 25

In an order-2 observer $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$*, consider* $\alpha \in (\Sigma_2)^* \cap \ell_2(L(G))$*, where* $\alpha \in (\Sigma_2)^*$ *leads* $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$ *to state* $\{(q_1, X_1), \ldots, (q_n, X_n)\}$*. Then* $\mathcal{M}_{A_1 \leftarrow A_2}(G) = \{X_1, \ldots, X_n\}$.

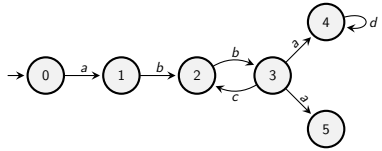
Theorem 25

In an order-2 observer $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$*, consider* $\alpha \in (\Sigma_2)^* \cap \ell_2(L(G))$*, where* $\alpha \in (\Sigma_2)^*$ *leads* $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$ *to state* $\{(q_1, X_1), \ldots, (q_n, X_n)\}$*. Then* $\mathcal{M}_{A_1 \leftarrow A_2}(G) = \{X_1, \ldots, X_n\}$*.*

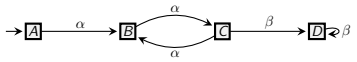
Theorem 26

An FSA G satisfies an order-2 estimation-based property PRED^2 (9) *with respect to agents* A_1 *and* A_2 *if and only if for every reachable state* $\{(q_1, X_1), \ldots, (q_n, X_n)\}$ *of the order-2 observer* $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$*,* $\{X_1, \ldots, X_n\} \in \mathrm{PRED}^2$*.*

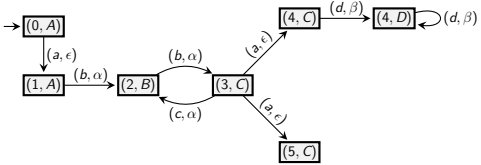
Example 27



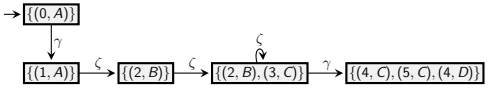
(a) Automaton G, where $E_1 = \{b, c, d\}$, $\ell_1(b) = \ell_1(c) = \alpha$, $\ell_1(d) = \beta$, $E_2 = \{a, b\}$, $\ell_2(a) = \gamma$, $\ell_2(b) = \zeta$.



(b) The observer Obs_{A_1} of automaton G_{A_1}, where $A = \{0, 1\}$, $B = \{2\}$, $C = \{3, 4, 5\}$, $D = \{4\}$.

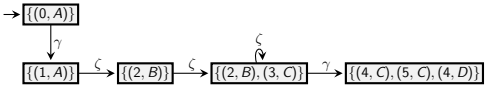


(c) The concurrent composition $\mathrm{CC}_{\ell_1}(G_{A_1}, \mathrm{Obs}_{A_1})$.



(d) The order-2 observer $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$.

Example 27 (cont'd)



Recall of $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$, where $A = \{0, 1\}$, $B = \{2\}$, $C = \{3, 4, 5\}$, $D = \{4\}$.

In $\mathrm{Obs}_{A_1 \leftarrow A_2}(G)$, there is a reachable state $\{(2, B)\}$ in which B is a singleton and there is no state in form of (q, X) with $|X| > 1$. By Thm. 26, G does not satisfy the order-2 estimation-based property

$$\mathrm{PRED}^2 = \{\emptyset \notin Y \subset 2^{\{0,1,2,3,4,5\}} | (\exists X \in Y)[|X| > 1]\} \subset 2^{2^{\{0,1,2,3,4,5\}}}.$$

An example of an order-2 estimation-based property

Define order-2 secrets as

$$Q_S^{\mathrm{ord}-2} \subset 2^Q. \tag{12}$$

Define predicate as

$$\mathrm{PRED}^2(G, Q_S^{\mathrm{ord}-2}) := \{\emptyset \notin Y \subset 2^Q | Y \not\subset Q_S^{\mathrm{ord}-2}\} \subset 2^{2^Q}. \tag{13}$$

An example of an order-2 estimation-based property

Define order-2 secrets as

$$Q_S^{\mathrm{ord}-2} \subset 2^Q. \tag{12}$$

Define predicate as

$$\mathrm{PRED}^2(G, Q_S^{\mathrm{ord}-2}) := \{\emptyset \notin Y \subset 2^Q | Y \not\subset Q_S^{\mathrm{ord}-2}\} \subset 2^{2^Q}. \tag{13}$$

Definition 28 (Ord2CSO)

An FSA G is called order-2 current-state opaque with respect to Usr, Intr, and $Q_S^{\mathrm{ord}-2}$ if

$$\{\mathcal{M}_{\mathrm{Usr}\leftarrow\mathrm{Intr}}(G, \alpha) | \alpha \in \ell_{\mathrm{Intr}}(L(G))\} \tag{14a}$$
$$\subset \mathrm{PRED}^2(G, Q_S^{\mathrm{ord}-2}). \tag{14b}$$

An example of an order-2 estimation-based property

Define order-2 secrets as

$$Q_S^{\text{ord}-2} \subset 2^Q. \tag{12}$$

Define predicate as

$$\text{PRED}^2(G, Q_S^{\text{ord}-2}) := \{\emptyset \notin Y \subset 2^Q | Y \not\subset Q_S^{\text{ord}-2}\} \subset 2^{2^Q}. \tag{13}$$

Definition 28 (Ord2CSO)

An FSA G is called order-2 current-state opaque with respect to Usr, Intr, and $Q_S^{\text{ord}-2}$ if

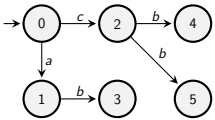
$$\{\mathcal{M}_{\text{Usr}\leftarrow\text{Intr}}(G, \alpha) | \alpha \in \ell_{\text{Intr}}(L(G))\} \tag{14a}$$

$$\subset \text{PRED}^2(G, Q_S^{\text{ord}-2}). \tag{14b}$$

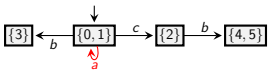
A special case of order-2 CSO

High-order opacity (Cui, Yin, S. Li, and Giua, 2022), where the methods are wrong. Counterexamples in (Zhang, Han, Giua, and Seatzu, 2024).

In (Cui, Yin, Li, and Giua, 2022), in order to verify high-order opacity (only the case consisting of two agents is considered.), a method called double-observer was proposed, but the method does not work correctly. The double-observer fails to compute $\mathcal{M}_{A_1 \leftarrow A_2}(G, \alpha)$.



(a) FSA G, where $E_{\text{Usr}} = \{b, c\}$, $E_{\text{Intr}} = \{a, b\}$, $\ell_{\text{Usr}}(b) = b$, $\ell_{\text{Usr}}(c) = c$, $\ell_{\text{Intr}}(a) = a$, $\ell_{\text{Intr}}(b) = b$.

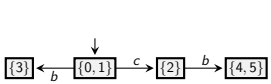


(b) The variant observer $\overline{\text{Obs}_{\text{Usr}}}$ of automaton G_{Usr}.

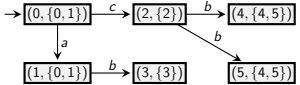


(c) The observer $\text{Obs}_{\text{Intr}}(\overline{\text{Obs}_{\text{Usr}}})$ of $\overline{\text{Obs}_{\text{Usr}}}$.

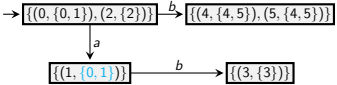
Figure 10: Illustration of the double-observer in (Cui, Yin, Li, and Giua, 2022).



(a) The observer Obs_{Usr} of automaton G_{Usr} (shown in 10a).



(b) The concurrent composition $\text{CC}^{G,\text{Obs}}_{\text{Usr}\to\text{Intr}}$.



(c) The order-2 observer $\text{Obs}_{\text{Usr}\leftarrow\text{Intr}}(G)$.

Figure 11: Illustration of order-2 observer $\text{Obs}_{\text{Usr}\leftarrow\text{Intr}}(G)$ in our paper.

Cui, B., X. Yin, S. Li, and A. Giua (2022). "You don't know what I know: on notion of high-order opacity in discrete-event systems". In: *IFAC-PapersOnLine* 55.28. 16th IFAC Workshop on Discrete Event Systems WODES 2022, pp. 135–141.

Content

1. Motivation
2. Review of basic tools — concurrent composition and observer
3. A system consisting of a single agent
   - A general framework
   - Case studies
4. A system consisting of two ordered agents
   - A general framework
   - Case studies
5. A system consisting of a finite number of ordered agents
   - A general framework
   - Case studies
6. Conclusion

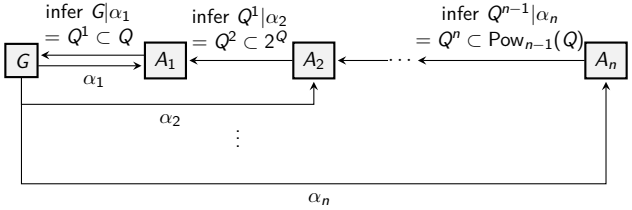


Figure 12: An illustration of order-n estimation-based property, where A_n infers what A_{n-1} knows about ... what A_2 knows about A_1's state estimate of G.

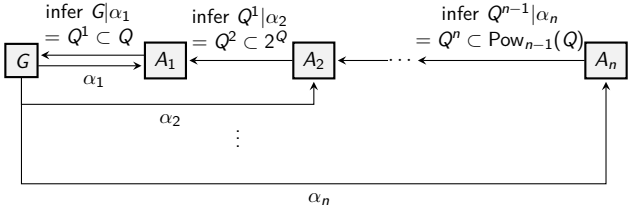


Figure 12: An illustration of order-n estimation-based property, where A_n infers what A_{n-1} knows about ... what A_2 knows about A_1's state estimate of G.

The setting

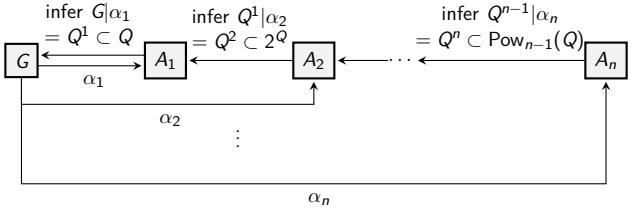


Figure 12: An illustration of order-n estimation-based property, where A_n infers what A_{n-1} knows about ... what A_2 knows about A_1's state estimate of G.

The setting

- G is known to $A_1, \ldots, A_n$.

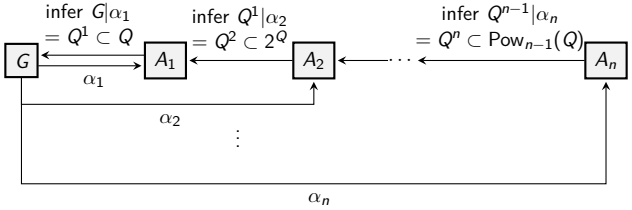


Figure 12: An illustration of order-n estimation-based property, where A_n infers what A_{n-1} knows about ... what A_2 knows about A_1's state estimate of G.

The setting

- G is known to $A_1, \ldots, A_n$.
- A_i observes $E_i \subset E$ via labeling function $\ell_i : E_i \to \Sigma_i$, $E \setminus E_i \to \{\epsilon\}$, $1 \leq i \leq n$.

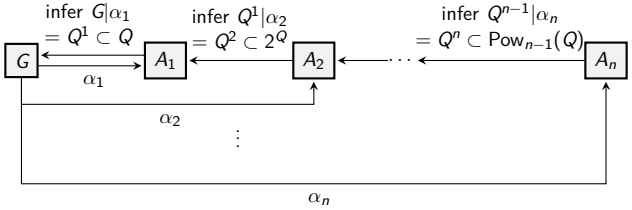


Figure 12: An illustration of order-n estimation-based property, where A_n infers what A_{n-1} knows about ... what A_2 knows about A_1's state estimate of G.

The setting

- G is known to $A_1, \ldots, A_n$.
- A_i observes $E_i \subset E$ via labeling function $\ell_i : E_i \to \Sigma_i$, $E \setminus E_i \to \{\epsilon\}$, $1 \leq i \leq n$.
- A_i knows E_{j_i} and ℓ_{j_i}, $1 \leq i \leq n$, $1 \leq j_i \leq i$.

Definition 29 (order-n predicate)

$$\mathrm{PRED}^n \subset \mathrm{Pow}_n(Q), \tag{15}$$

where $\mathrm{Pow}_i(Q) = \mathrm{Pow}(\mathrm{Pow}_{i-1}(Q))$, $i \in [\![2, n]\!]$, $\mathrm{Pow}_1(Q) = 2^Q$.

For $\alpha \in \ell_n(L(G))$,

$$
\begin{aligned}
&\mathcal{M}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G, \alpha) \\
:= &\overbrace{\{\{\ldots\{}^{n-1} \mathcal{M}_1(G, \ell_1(s_2))| \\
&s_2 \in \ell_1^{-1}(\ell_2(s_3)) \cap L(G)\}| \\
&\ldots \\
&s_{n-1} \in \ell_{n-2}^{-1}(\ell_{n-1}(s_n)) \cap L(G)\}| \\
&s_n \in \ell_n^{-1}(\alpha) \cap L(G)\} \\
\subset &\mathrm{Pow}_{n-1}(Q).
\end{aligned} \tag{16}
$$

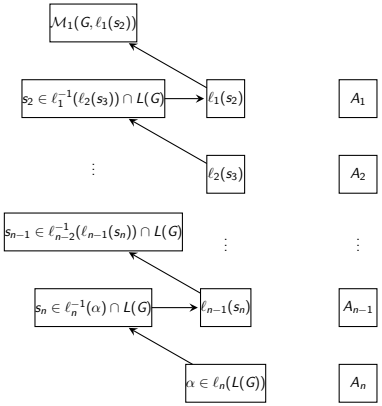


Figure 13: Illustration of order-n current-state estimate $\mathcal{M}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G, \alpha)$ of G with respect to $\alpha \in \ell_n(L(G))$.

Definition 30

An FSA G satisfies the order-n estimation-based property PRED^n with respect to agents $A_1, \ldots, A_n$ if

$$\{\mathcal{M}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G, \alpha) | \alpha \in \ell_n(L(G))\} \subset \text{PRED}^n. \tag{17}$$

Definition 30

An FSA G satisfies the order-n estimation-based property PRED^n with respect to agents $A_1, \ldots, A_n$ if

$$\{\mathcal{M}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G, \alpha) | \alpha \in \ell_n(L(G))\} \subset \text{PRED}^n. \tag{17}$$

Definition 31

Consider an FSA G and agents A_i with observable event sets $E_i \subset E$, $i \in [\![1, n]\!]$. The order-1 observer is defined as Obs_{A_1}. For each $n > 1$, the order-n observer $\text{Obs}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G)$ is defined as

$$\text{Obs}(\text{CC}_{A_1 \to A_2 \to \cdots \to A_{n-1} \to A_n}^{G, \text{Obs}(G, \text{Obs}(\ldots(G, \text{Obs}\overbrace{)\ldots)}^{n-2})}).$$

Definition 30

An FSA G satisfies the order-n estimation-based property PRED^n with respect to agents $A_1, \ldots, A_n$ if

$$\{\mathcal{M}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G, \alpha) | \alpha \in \ell_n(L(G))\} \subset \text{PRED}^n. \tag{17}$$

Definition 31

Consider an FSA G and agents A_i with observable event sets $E_i \subset E$, $i \in [\![1, n]\!]$. The order-1 observer is defined as Obs_{A_1}. For each $n > 1$, the order-n observer $\text{Obs}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G)$ is defined as

$$\text{Obs}(\text{CC}_{A_1 \to A_2 \to \cdots \to A_{n-1} \to A_n}^{G, \text{Obs}(G, \text{Obs}(\ldots(G, \text{Obs}\overbrace{)\ldots)}^{n-2})}).$$

Definition 30

An FSA G satisfies the order-n estimation-based property PRED^n with respect to agents $A_1, \ldots, A_n$ if

$$\{\mathcal{M}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G, \alpha) | \alpha \in \ell_n(L(G))\} \subset \text{PRED}^n. \tag{17}$$

Definition 31

Consider an FSA G and agents A_i with observable event sets $E_i \subset E$, $i \in [\![1, n]\!]$. The order-1 observer is defined as Obs_{A_1}. For each $n > 1$, the order-n observer $\text{Obs}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G)$ is defined as

$$\text{Obs}(\text{CC}_{A_1 \to A_2 \to \cdots \to A_{n-1} \to A_n}^{G, \text{Obs}(G, \text{Obs}(\ldots(G, \text{Obs}\overbrace{)\ldots)}^{n-2})}).$$

1. An order-n observer is computable in n-EXPTIME.

Definition 30

An FSA G satisfies the order-n estimation-based property PRED^n with respect to agents $A_1, \ldots, A_n$ if

$$\{\mathcal{M}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G, \alpha) | \alpha \in \ell_n(L(G))\} \subset \text{PRED}^n. \tag{17}$$

Definition 31

Consider an FSA G and agents A_i with observable event sets $E_i \subset E$, $i \in [\![1, n]\!]$. The order-1 observer is defined as Obs_{A_1}. For each $n > 1$, the order-n observer $\text{Obs}_{A_1 \leftarrow A_2 \leftarrow \cdots \leftarrow A_n}(G)$ is defined as

$$\text{Obs}(\text{CC}^{G, \text{Obs}(G, \text{Obs}(\ldots(G, \overbrace{\text{Obs}}^{n-2})\ldots)}_{A_1 \to A_2 \to \cdots \to A_{n-1} \to A_n}).$$

1. An order-n observer is computable in n-EXPTIME.
2. Use the order-n observer to verify an order-n estimation-based property in n-EXPTIME.

I know you don't know what I know

Definition 32

Consider an FSA G, user Usr and intruder Intr with observable event sets $E_{\text{Usr}} \subset E$ and $E_{\text{Intr}} \subset E$, respectively. G satisfies the order-3 current-state opacity with respect to Usr, Intr, and Usr if for all $\alpha \in \ell_{\text{Usr}}(L(G))$, for all $Y \in \mathcal{M}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G, \alpha) \subset 2^{2^Q}$, $Y \not\subset \{\{q\} | q \in Q\}$.

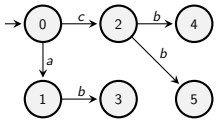
I know you don't know what I know

Definition 32

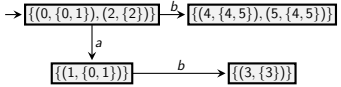
Consider an FSA G, user Usr and intruder Intr with observable event sets $E_{\text{Usr}} \subset E$ and $E_{\text{Intr}} \subset E$, respectively. G satisfies the order-3 current-state opacity with respect to Usr, Intr, and Usr if for all $\alpha \in \ell_{\text{Usr}}(L(G))$, for all $Y \in \mathcal{M}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G, \alpha) \subset 2^{2^Q}$, $Y \not\subset \{\{q\} | q \in Q\}$.

Order-3 current-state opacity means that Usr knows that Intr does not know that Usr can uniquely determine the current state of G.

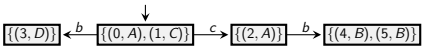
Example 33



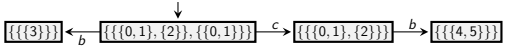
(a) FSA G, where $E_{\text{Usr}} = \{b, c\}$, $E_{\text{Intr}} = \{a, b\}$, $\ell_{\text{Usr}}(b) = b$, $\ell_{\text{Usr}}(c) = c$, $\ell_{\text{Intr}}(a) = a$, $\ell_{\text{Intr}}(b) = b$.



(b) The order-2 observer $\text{Obs}_{\text{Usr}\leftarrow\text{Intr}}(G)$.



(c) Order-3 observer $\text{Obs}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G)$, where $A = \{(0, \{0, 1\}), (2, \{2\})\}$, $B = \{(4, \{4, 5\}), (5, \{4, 5\})\}$, $C = \{(1, \{0, 1\})\}$, $D = \{(3, \{3\})\}$.



(d) Obtained from order-3 observer $\text{Obs}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G)$, Usr's inference of the inference of Intr to Usr's state estimate to G.

$$\mathcal{M}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G, \epsilon) = \{\{\{0, 1\}, \{2\}\}, \{\{0, 1\}\}\}, \tag{18a}$$

$$\mathcal{M}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G, c) = \{\{\{0, 1\}, \{2\}\}\}, \tag{18b}$$

$$\mathcal{M}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G, cb) = \{\{\{4, 5\}\}\}, \tag{18c}$$

$$\mathcal{M}_{\text{Usr}\leftarrow\text{Intr}\leftarrow\text{Usr}}(G, b) = \{\{\{3\}\}\}. \tag{18d}$$

Example 33 (cont'd)

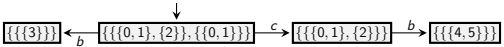


Figure 15: Obtained from order-3 observer $\mathrm{Obs}_{\mathsf{Usr}\leftarrow\mathsf{Intr}\leftarrow\mathsf{Usr}}(G)$, Usr's inference of the inference of Intr to Usr's state estimate to G.

$$\mathcal{M}_{\mathsf{Usr}\leftarrow\mathsf{Intr}\leftarrow\mathsf{Usr}}(G, b) = \{\{\{3\}\}\}. \tag{18d}$$

By $\mathrm{Obs}_{\mathsf{Usr}}$, $\mathcal{M}_{\mathsf{Usr}}(G, b) = \{3\}$, that is, by observing b, Usr uniquely determines the current state of G. Then by the order-3 observer and (18d), Usr knows that Intr exactly knows $\mathcal{M}_{\mathsf{Usr}}(G, b)$. Hence system G is not sufficiently safe for Usr to operate on.

Content

1. Motivation
2. Review of basic tools — concurrent composition and observer
3. A system consisting of a single agent
   - A general framework
   - Case studies
4. A system consisting of two ordered agents
   - A general framework
   - Case studies
5. A system consisting of a finite number of ordered agents
   - A general framework
   - Case studies
6. Conclusion

- Formulation and verification of "know what" in a sequence of labeled finite-state automata (LFSAs).

- Formulation and verification of "know what" in a sequence of labeled finite-state automata (LFSAs).
- Complexity: n-EXPTIME when there are n agents.

- Formulation and verification of "know what" in a sequence of labeled finite-state automata (LFSAs).
- Complexity: n-EXPTIME when there are n agents.
- For special predicates, complexity can be reduced:

- Formulation and verification of "know what" in a sequence of labeled finite-state automata (LFSAs).
- Complexity: n-EXPTIME when there are n agents.
- For special predicates, complexity can be reduced:
  - For each i, $E_i \subset E_{i+1}$ or $E_i \supset E_{i+1}$, all labeling functions are projections, in EXPTIME.

- Formulation and verification of "know what" in a sequence of labeled finite-state automata (LFSAs).
- Complexity: n-EXPTIME when there are n agents.
- For special predicates, complexity can be reduced:
  - For each i, $E_i \subset E_{i+1}$ or $E_i \supset E_{i+1}$, all labeling functions are projections, in EXPTIME.
  - Induced by a 2-bounded order-1 predicate, in $(n-1)$-EXPTIME.

- Formulation and verification of "know what" in a sequence of labeled finite-state automata (LFSAs).
- Complexity: n-EXPTIME when there are n agents.
- For special predicates, complexity can be reduced:
  - For each i, $E_i \subset E_{i+1}$ or $E_i \supset E_{i+1}$, all labeling functions are projections, in EXPTIME.
  - Induced by a 2-bounded order-1 predicate, in $(n-1)$-EXPTIME.

Outlook

Formulation and verification for other fragments in epistemic logic in LFSAs or other types of extended automata.

Thank you!

References I

Shu, S. and F. Lin (2011). "Generalized detectability for discrete event systems". In: *Systems & Control Letters* 60.5, pp. 310–317.

Jiang, S., Z. Huang, V. Chandra, and R. Kumar (Aug. 2001). "A polynomial algorithm for testing diagnosability of discrete-event systems". In: *IEEE Transactions on Automatic Control* 46.8, pp. 1318–1321.

Yoo, T.-S. and S. Lafortune (Sept. 2002). "Polynomial-time verification of diagnosability of partially observed discrete-event systems". In: *IEEE Transactions on Automatic Control* 47.9, pp. 1491–1495.

Genc, S. and S. Lafortune (2009). "Predictability of event occurrences in partially-observed discrete-event systems". In: *Automatica* 45.2, pp. 301–311.

Zhang, K. (2023a). "A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems". In: *Annual Reviews in Control* 56, 100902(1–21).

Ramadge, P.J. and W.M. Wonham (1987). "Supervisory control of a class of discrete event processes". In: *SIAM Journal on Control and Optimization* 25.1, pp. 206–230.

Shu, S., F. Lin, and H. Ying (Dec. 2007). "Detectability of discrete event systems". In: *IEEE Transactions on Automatic Control* 52.12, pp. 2356–2359.

Sampath, M., R. Sengupta, S. Lafortune, K. Sinnamohideen, and D. Teneketzis (Sept. 1995). "Diagnosability of discrete-event systems". In: *IEEE Transactions on Automatic Control* 40.9, pp. 1555–1575.

References II

Saboori, A. and C.N. Hadjicostis (July 2011). "Verification of K-step opacity and analysis of its complexity". In: *IEEE Transactions on Automation Science and Engineering* 8.3, pp. 549–559.

— (Dec. 2007). "Notions of security and opacity in discrete event systems". In: *46th IEEE Conference on Decision and Control*, pp. 5056–5061.

Wu, Y. and S. Lafortune (Sept. 2013). "Comparative analysis of related notions of opacity in centralized and coordinated architectures". In: *Discrete Event Dynamic Systems* 23.3, pp. 307–339.

Saboori, A. and C.N. Hadjicostis (2009). "Verification of K-step opacity and analysis of its complexity". In: *Proceedings of the 48h IEEE Conference on Decision and Control (CDC) held jointly with 2009 28th Chinese Control Conference*, pp. 205–210.

Yin, X. and S. Lafortune (2017). "A new approach for the verification of infinite-step and K-step opacity using two-way observers". In: *Automatica* 80, pp. 162–171.

Ma, Z., X. Yin, and Z. Li (2021). "Verification and enforcement of strong infinite- and K-step opacity using state recognizers". In: *Automatica* 133, p. 109838.

Wintenberg, A., M. Blischke, S. Lafortune, and N. Ozay (2022). "A general language-based framework for specifying and verifying notions of opacity". In: *Discrete Event Dynamic Systems* 32.2, pp. 253–289.

Zhang, K. (2023b). "A new framework for discrete-event systems". In: *Foundations and Trends® in Systems and Control* 10.1-2, pp. 1–179.

References III

Zhang, K. and A. Giua (2020). "On detectability of labeled Petri nets and finite automata". In: *Discrete Event Dynamic Systems* 30.3, pp. 465–497.

Rabin, M.O. and D. Scott (1959). "Finite automata and their decision problems". In: *IBM Journal of Research and Development* 3.2, pp. 114–125.

Özveren, C.M. and A.S. Willsky (July 1990). "Observability of discrete event dynamic systems". In: *IEEE Transactions on Automatic Control* 35.7, pp. 797–806.

Cassez, F., J. Dubreil, and H. Marchand (2009). "Dynamic observers for the synthesis of opaque systems". In: *Automated Technology for Verification and Analysis*. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 352–367.

Lin, F. (2011). "Opacity of discrete event systems and its applications". In: *Automatica* 47.3, pp. 496–503.

Pola, G., E. De Santis, M.D. Di Benedetto, and D. Pezzuti (2017). "Design of decentralized critical observers for networks of finite state machines: a formal method approach". In: *Automatica* 86, pp. 174–182.

Han, X., K. Zhang, J. Zhang, Z. Li, and Z. Chen (2023). "Strong current-state and initial-state opacity of discrete-event systems". In: *Automatica* 148, 110756(1–8).

Cui, B., X. Yin, S. Li, and A. Giua (2022). "You don't know what I know: on notion of high-order opacity in discrete-event systems". In: *IFAC-PapersOnLine* 55.28. 16th IFAC Workshop on Discrete Event Systems WODES 2022, pp. 135–141.

References IV



Zhang, K., X. Han, A. Giua, and C. Seatzu (2024). "High-order observers and high-order state-estimation-based properties of discrete-event systems".